

# **Reti Wireless e Mobili**

# Indice

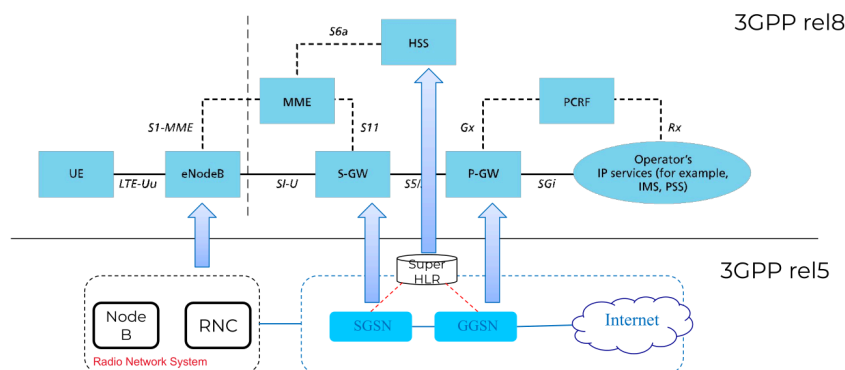
|                                                |           |
|------------------------------------------------|-----------|
| <b>1. Lezione 11 [23/02]</b>                   | <b>4</b>  |
| 1.0.1. LTE (4G)                                | 4         |
| 1.0.1.1. Rete Core                             | 5         |
| 1.0.1.1.1. MME                                 | 5         |
| 1.0.1.1.2. HSS                                 | 5         |
| 1.0.1.1.3. PDNGW                               | 5         |
| 1.0.1.1.4. SGW                                 | 5         |
| 1.0.1.1.5. PCRF                                | 6         |
| 1.0.1.2. E-UTRAN                               | 6         |
| 1.0.1.2.1. Trasmissione                        | 6         |
| 1.0.1.2.2. Resource Block                      | 8         |
| 1.0.1.2.3. Velocità                            | 10        |
| 1.0.1.2.4. Frequenze                           | 10        |
| <b>2. Lezione 12 [27/02]</b>                   | <b>11</b> |
| 2.1. Rete cellulare                            | 11        |
| 2.1.1. E-UTRAN                                 | 11        |
| 2.1.1.1. Architettura                          | 11        |
| 2.1.1.2. Tracking area                         | 11        |
| 2.1.1.3. Interfaccia X2                        | 12        |
| 2.1.1.4. Architettura ma più densa             | 12        |
| 2.1.1.5. Control Plane                         | 12        |
| 2.1.1.5.1. SCTP                                | 13        |
| 2.1.1.6. User Plane                            | 15        |
| 2.1.1.7. Bearer                                | 17        |
| 2.1.1.8. Collegamento alla rete dell'operatore | 18        |
| <b>3. Lezione 13 [02/03]</b>                   | <b>21</b> |
| 3.1. Rete cellulare                            | 21        |
| 3.1.1. LTE                                     | 21        |
| 3.1.1.1. Handover                              | 21        |
| 3.1.1.1.1. Interfaccia S1                      | 21        |
| 3.1.1.1.2. Interfaccia X2                      | 22        |
| 3.1.2. Edge Computing e Network Softwarization | 22        |
| 3.1.2.1. SDN                                   | 23        |
| 3.1.2.2. NFV                                   | 23        |
| 3.1.2.3. Cloud                                 | 24        |
| <b>4. Lezione14 [06/03]</b>                    | <b>25</b> |
| 4.1. 5G                                        | 25        |
| 4.1.1. NFV su RAN                              | 25        |
| 4.1.2. Cloud/Edge computing                    | 25        |
| 4.1.3. Network Slices                          | 26        |
| 4.1.4. Architettura                            | 26        |
| 4.1.4.1. Control plane                         | 27        |
| 4.1.4.2. User plane                            | 27        |
| <b>5. Lezione 15 [10/03]</b>                   | <b>29</b> |
| 5.1. 5G                                        | 29        |

|                                                       |    |
|-------------------------------------------------------|----|
| 5.1.1. Network Slice .....                            | 29 |
| 5.1.2. Integrazione con Edge Computing .....          | 29 |
| 5.1.3. Latenza .....                                  | 29 |
| 5.1.4. Architettura Standalone e Non-Standalone ..... | 30 |
| 5.2. Rete satellitare .....                           | 30 |
| 5.2.1. Orbite .....                                   | 30 |
| 5.2.2. Architettura .....                             | 31 |
| 5.2.3. NTN .....                                      | 31 |

# 1. Lezione 11 [23/02]

## 1.0.1. LTE (4G)

Vediamo la rete **Long Term Evolution (LTE) 4G**, che viene resa disponibile nella release 8.



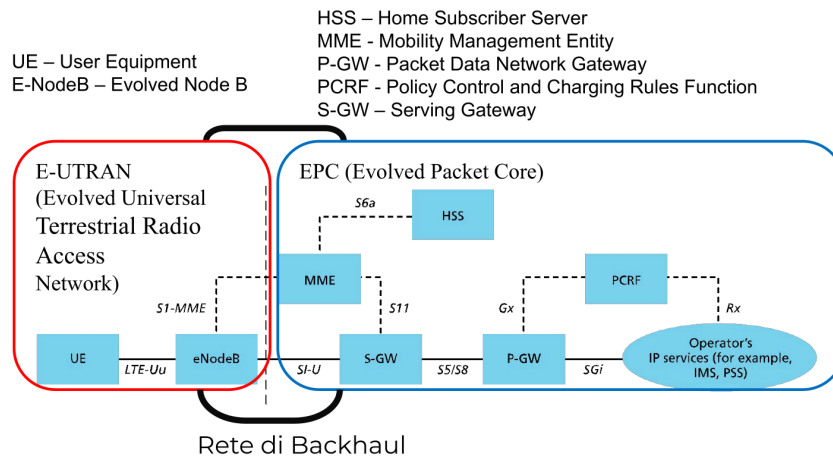
Come vediamo, non cambiamo molto architetturalmente parlando: vediamo solo una fusione di **NodeB** e **RNC** in una unica **entità eNodeB** e un cambio di nome di alcuni **gateway**. Inoltre, qua si fa solo **hard handover**.

Vediamo le **differenze** principali tra **3G** e **4G**.

|                                         | 3G                       | 4G                        |
|-----------------------------------------|--------------------------|---------------------------|
| Separazione logica Control e Data plane | No                       | Sì                        |
| Accesso multiplo                        | WCDMA                    | OFDMA                     |
| Riuso frequenze                         | 100%                     | 100% flessibile           |
| Larghezza banda                         | 5 MHz                    | 1.4, 3, 5, 10, 15, 20 MHz |
| Componenti rete accesso                 | NodeB e RNC              | eNodeB                    |
| Handover                                | Soft e Hard              | Hard                      |
| Trasporto                               | Circuito e Packet Switch | Packet Switch             |
| Servizi voce e SMS                      | Interni alla rete        | Esterni alla rete         |

Prima i moduli potevano essere sia controllo che dati, mentre ora abbiamo una **separazione netta**, con moduli che fanno solo dati o solo controllo. In **5G** vedremo che questa cosa sarà super esagerata.

Vediamo la **divisione logica** di questa **architettura** e andiamo a studiarla.



Il blocco di sinistra è la **Evolved UTRAN** (E-UTRAN), il blocco centrale è la **rete di backhaul** (connessione tra BS e rete core) e il blocco di destra è la **rete core**.

#### 1.0.1.1. Rete Core

Partiamo con la **rete core**. Il blocco per ora più semplice è quello dei **servizi operatore**, che sono esterni alla rete.

##### 1.0.1.1.1. MME

La **Mobile Management Entity** (MME) è un nodo che si occupa del solo **traffico di controllo**, quindi **NAS**, e gestisce:

1. contesto dell'**User Equipment** (UE) tramite operazioni NAS;
2. **bearer**, come controllo, autorizzazione, creazione, mantenimento, distruzione, eccetera;
3. **mobilità** all'intero della tracking area;
4. **paging**
5. aspetti di sicurezza e cifratura.

##### 1.0.1.1.2. HSS

L'**Home Subscribe Server** (HSS) contiene le informazioni dell'utente e dell'abbonamento, come una sorta di database. Per essere precisi contiene:

1. profili di QoS ammessi;
2. restrizioni roaming;
3. informazioni **APN**, ovvero gli IP dei singoli **PDNGW**;
4. identità dell'MME a cui un UE è registrato.

##### 1.0.1.1.3. PDNGW

Il **Packed Data Network Gateway** (PDNGW) è il ponte verso il mondo esterno, che:

1. assegna un IP ad ogni UE;
2. garantisce le **QoS policy**;
3. filtra i pacchetti **IP DL** in bearer differenti;
4. gestisce la mobilità tra reti non-3GPP.

##### 1.0.1.1.4. SGW

Il **Serving Gateway** (SGW) gestisce il **traffico user plane**. In particolare:

1. gestisce tutti i pacchetti nella rete dell'operatore;
2. fa da **ancora mobile** quando si è in handover;
3. **bufferizza** quando un UE è in IDLE-CONNECTED.

#### 1.0.1.1.5. PCRF

Il **Policy Control and Charging Rules Function** (PCRF) svolge:

1. controllo e autorizzazione di singoli flussi a livello PGW;
2. autorizza i QoS secondo i profili utenti dall'HSS.

#### 1.0.1.2. E-UTRAN

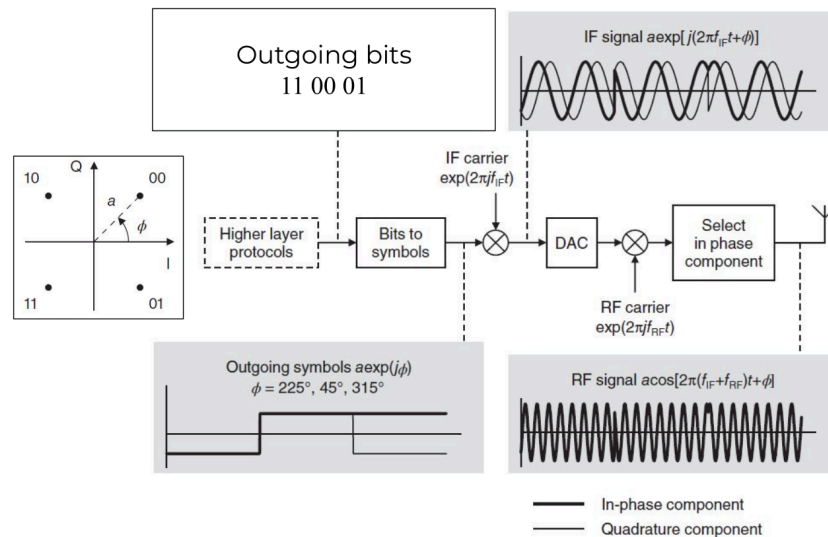
Nel **E-UTRAN** abbiamo solo la **BS**, che è detta **eNodeB**. Questa dà connessione agli UE e li collega alla rete network, quindi fa sia controllo che dati.

In particolare, deve:

1. gestire le risorse radio e l'accesso al canale tramite OFDMA;
2. fare compressione delle risorse radio;
3. fare connessione con SGW e MME per traffico dati e controllo;
4. dare informazioni posizione UE;
5. dare sicurezza e crittografia al canale radio.

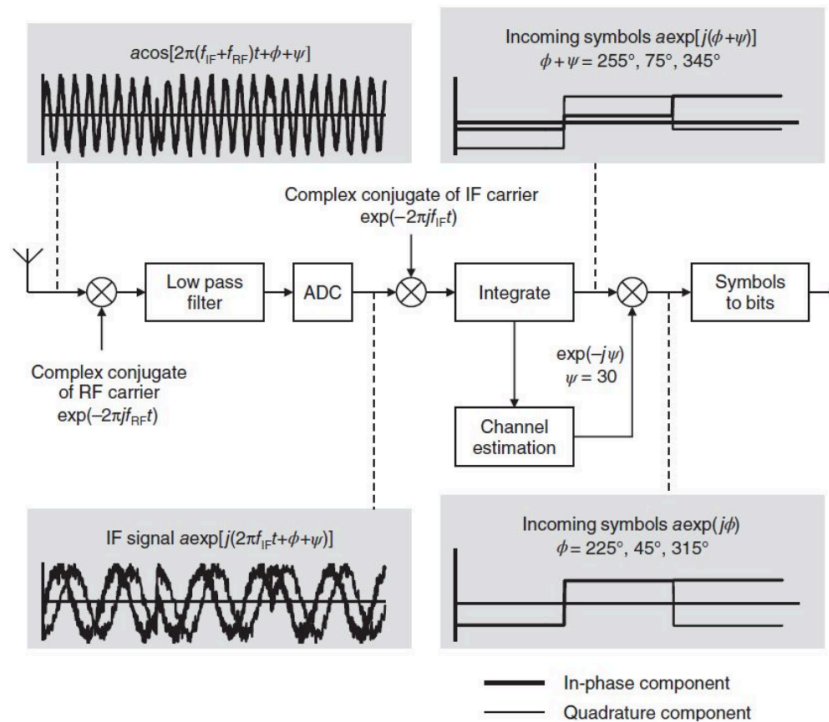
##### 1.0.1.2.1. Trasmissione

Siamo nella **E-UTRAN**, vediamo come avviene la **trasmissione** con **QPSK**.



Come vediamo, dopo essere passati **dai bit ai simboli**, guardiamo la costellazione e moduliemo su una **frequenza intermedia IF** di OFDMA, con tutte le sue sotto-portanti. Infine, dopo il **Digital to Analog Converter** (DAC) portiamo tutto in banda traslata e trasmettiamo.

Vediamo ora la fase di **ricezione**, che sicuramente vedrà un'onda distorta visto il rumore termico e lo sfasamento indotto dalla mobilità  $\psi$ .



In questo caso torniamo in banda base, filtriamo per rimuovere il rumore e convertiamo con l'**Analog to Digital Converter** (ADC). A questo punto non abbiamo la fase  $\phi$  trasmessa ma  $\phi + \psi$  che è distorta.

Quello che facciamo è una **channel estimation** tramite **pilot**, che essendo segnali standard possono essere confrontati e usati per capire il fattore di errore da togliere come fase.

Avviene poi la trasformazione da simboli a bit e il gioco è fatto.

Questa soluzione di **channel estimation** è usata anche per capire se si deve fare o meno un **handover**.

Nella rete E-UTRAN si usano quattro diverse modulazioni:

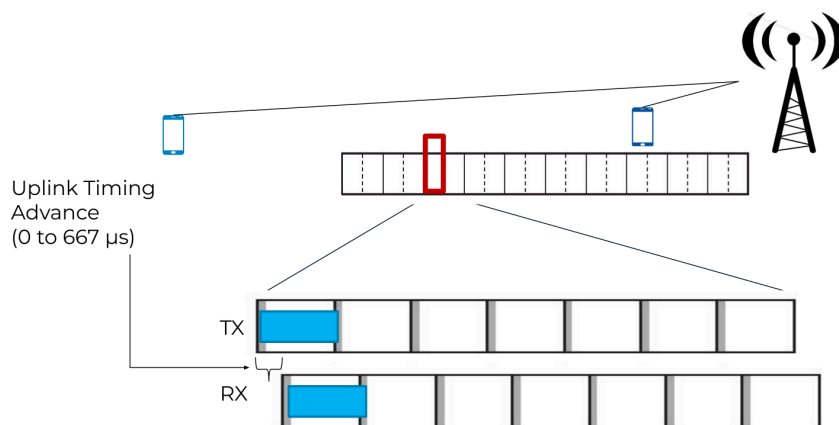
1. **BPSK** (binary), usata per segnali a basso livello mandati dalla BS per comunicare come funziona la BS stessa, il duplex, eccetera. Infatti, devono essere **facilmente decodificabili** anche a fronte di un canale pessimo;
2. **QPSK** (quadrature), usata per controllo e trasmissione se abbiamo scarsa qualità del segnale;
3. **16/64-QAM**, usata per la trasmissione dati.





Viene aggiunto un **periodo di guardia** dopo ogni DL e un inizio di UL per permettere l'**Uplink Timing Advance**.

Infatti, può avvenire che un dispositivo lontano inizi a trasmettere sfiorando il prefisso ciclico. Viene quindi detto dalla BS di **quanto anticipare** la trasmissione per arrivare in tempo con la trasmissione. Inoltre, visto questo anticipo abbiamo anche un periodo di vuoto della BS così da evitare l'**interferenza**.

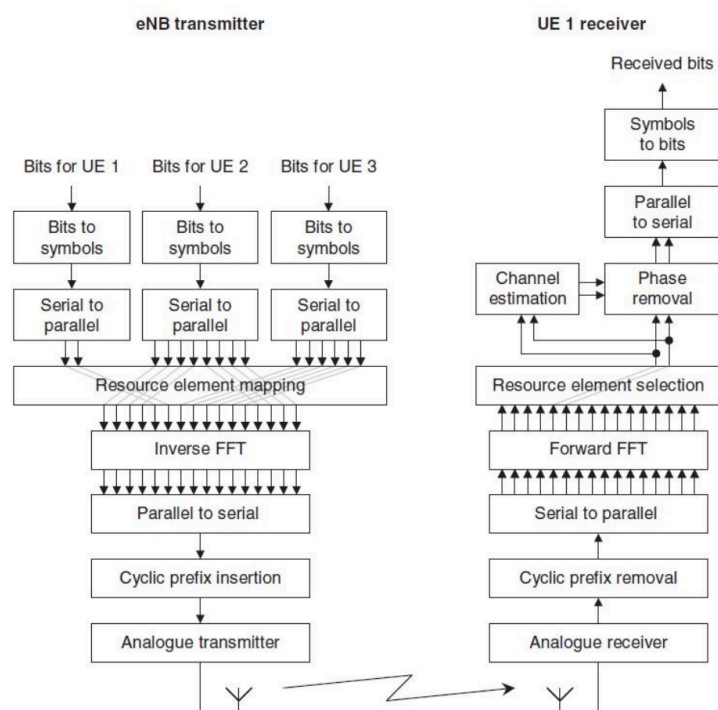


Definiamo ora le **risorse minime** che possiamo dare agli utenti.

I **Resource Block** sono come le RU di WiFi6, ma qui siamo più **flessibili**, visto che dobbiamo garantire **flessibilità di sistema**.

Usiamo **OFDMA**, dividendo la banda in sotto-portanti, avendo quindi simboli più lunghi e distanza tra sotto-portanti più bassa. Più sotto-bande sono organizzate in questi **Resource Block**, che sono la minima quantità di risorse allocabili.

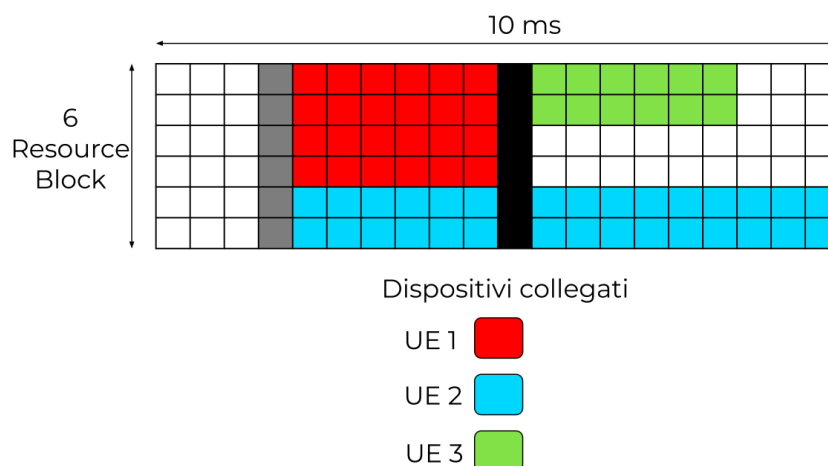
Sappiamo benissimo come funziona **OFDMA**, ma qua dobbiamo assegnare le sotto-portanti agli utenti, quindi è un pelo diversa la situazione.



In questo caso in trasmissione ci serve il **resource element mapping**, che mappa i vari dispositivi nelle loro sotto-portanti. In ricezione abbiamo invece il **resource element selection**, che fa una **equalizzazione**, un **filtro**, ottenendo solo le sotto-portanti che ci sono state assegnate.

Ogni **Resource Block** è un blocco di 12 sotto-bande, ognuna di 7 simboli. Ogni **eNodeB** deve essere in grado di garantire **6 RB** per 10ms, quindi un totale di 20 colonne.

Gli **eNodeB** poi hanno uno **scheduler** che sceglie chi allocare e dove.



#### 1.0.1.2.3. Velocità

La **velocità** che offre la rete LTE dipende da molti fattori:

1. capacità del dispositivo;
2. qualità del segnale radio, che determina la codifica;
3. larghezza della banda, che determina il numero di RB;
4. configurazione del TDD;
5. numero di dispositivi collegati;
6. congestione delle rete backhaul e dei PGW.

Le massime velocità teoriche sono:

1. 300Mbps in **download**;
2. 75Mbps in **upload**.

#### 1.0.1.2.4. Frequenze

Finiamo la parte **E-UTRAN** vedendo come sono allocate le frequenze in Italia nel 2011.

| Banda   | Numero lotti | Ampiezza (MHz) | Modalità Duplex |
|---------|--------------|----------------|-----------------|
| 800MHz  | 6            | 5MHz           | FDD             |
| 1800MHz | 3            | 5MHz           | FDD             |
| 2000MHz | 1            | 15MHz          | TDD             |
| 2600MHz | 12           | 5MHz           | FDD             |
|         | 2            | 15MHz          | TDD             |

Ognuno di questi lotti è pagato a peso d'oro, visto che si hanno pochi lotti ma ognuno di questo permette una scalabilità immensa.

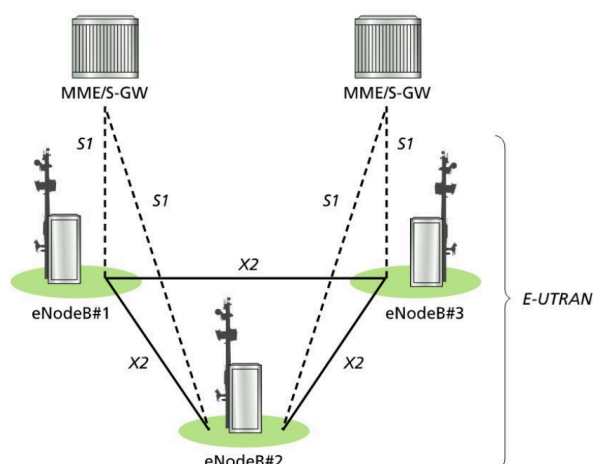
## 2. Lezione 12 [27/02]

### 2.1. Rete cellulare

#### 2.1.1. E-UTRAN

##### 2.1.1.1. Architettura

Abbandoniamo il livello fisico e vediamo in generale l'**architettura** della rete RAN, che **interfacce** ci sono e che **protocolli** hanno a bordo i dispositivi.



In questa **architettura** i «fili» sono **interfacce logiche**, sulle quali abbiamo dei **protocolli** specifici che ci dicono cosa può viaggiare.

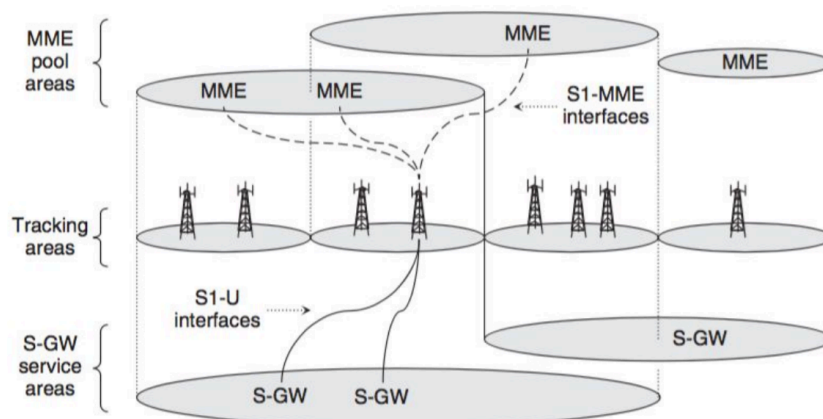
Le linee tratteggiate ci collegano alla rete core tramite **interfaccia S1**, mentre le linee continue permettono agli eNodeB di **parlare tra loro** senza interpellare la rete core usando il **protocollo X2**.

Infatti, questa è una innovazione di LTE, che permette ai eNodeB di fare **auto config**, gestire l'**handover** e regolare la **potenza di trasmissione**.

Ovviamente, sono **connessioni logiche**, quindi possono essere **realizzate** in vari modi, usando ponti radio oppure usando la fibra verso la rete IP classica.

##### 2.1.1.2. Tracking area

Gli **eNodeB** sono raggruppati in aree geografiche, e ogni eNodeB viene gestito da un **pool di MME**, che sono nodi di rete core che permettono di fare load balancing e fault tolerance.



Queste aree geografiche sono le **tracking area**: abbiamo gruppi di SGW e MME che gestiscono le varie aree, tramite **associazione dinamica**, permettendo di fare del **load balancing** (gestione del carico di rete) e **fault tolerance** (se un MME cade ci sono altri a coprirlo).

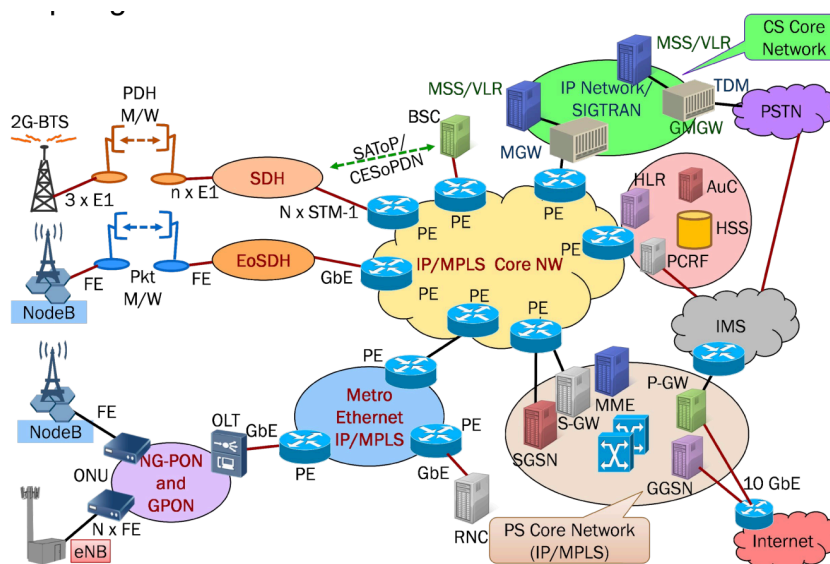
### 2.1.1.3. Interfaccia X2

L'**interfaccia X2** permette la **comunicazione diretta tra eNodeB**, e questo ci permette di:

1. gestire l'**handover** in alternativa al protocollo S1;
2. avere le **Self-Organizing Network** (SON) facendo load balancing e gestione delle interferenze (potenza di trasmissione);
3. mantenere uno **storico** delle ultime celle visitate per gestire l'effetto ping-pong tra le celle, negando l'handover se uno zio vuole fare troppi handover di fila.

### 2.1.1.4. Architettura ma più densa

Purtroppo, quando vediamo il link dalla rete E-UTRAN alla rete core, questo non è un **semplice hop** ma è un percorso ben più complicato.

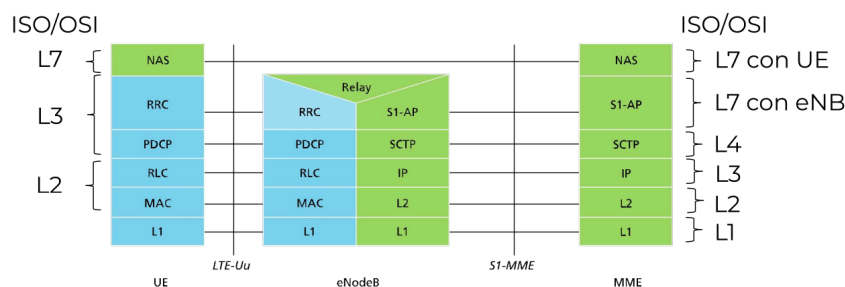


Come vediamo, noi siamo in basso a sinistra con la **rete E-UTRAN**, ma per arrivare alla **rete core** in basso a destra dobbiamo passare per un sacco di reti, che magari sono pure intasate.

Quindi, ovviamente, abbiamo molto **delay** e non un singolo hop come di solito facciamo vedere.

### 2.1.1.5. Control Plane

Vediamo come funziona il **Control Plane**.



Al **livello fisico** abbiamo RAN a sinistra e quello che vogliamo a destra. Al **livello MAC** abbiamo invece lo scheduler dei RB. Sopra questi abbiamo alcuni **protocolli nuovi**, che ora vediamo.

Partiamo con i protocolli **tra UE ed eNodeB**.

Il **protocollo Radio Resource Control (RRC)** gestisce le risorse radio, ma non il link radio. Si occupa di **paging**, **mobilità** (handover) e raccoglie le misurazioni degli UE per la QoS.

Il **Packet Data Convergence Protocol (PDCP)** permette la compressione degli header, mappando i dati sui canali fisici sottostanti.

Il **protocollo Radio Link Control (RLC)** si occupa di controllare il link al livello due, facendo correzione degli errori, gestione della ritrasmissione e segmentazione/assemblaggio dei pacchetti dei livelli superiori mettendoli in blocchi MAC.

Il **protocollo Medium Access Control (MAC)** classico gestisce l'accesso al canale radio e gestisce lo scheduler dei RB, ma solo nell'eNodeB.

Vediamo ora i protocolli **tra eNodeB e MME**.

Il **protocollo S1-AP** si trova al livello applicazione per permettere il trasporto del **traffico di controllo**.

Lo **Stream Control Transmission Protocol (SCTP)** si trova al livello trasporto e viene usato per ottimizzare il trasporto del traffico di segnalazione. Lo vedremo tra poco nello specifico.

Il **protocollo IP** riguarda gli indirizzi **interni**, privati dentro la rete dell'operatore, e fa routing dei messaggi del control plane.

Ovviamente, l'eNodeB è **dual stack**, visto che deve parlare radio con gli UE e cablato con l'MME.

Infine, **tra UE e MME** abbiamo solo il **protocollo Non-Access Stratum (NAS)**, che permette messaggi di livello 7 direttamente tra UE e MME usando gli eNodeB come **relay** (ripetitore).

#### **2.1.1.5.1. SCTP**

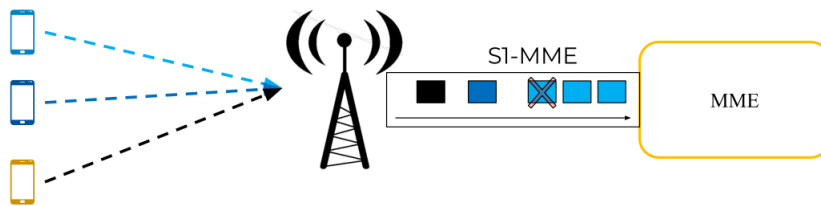
Diamo un occhio nello specifico al **protocollo SCTP**, e perché è stato scelto questo e non il classico protocollo TCP.

Il **protocollo TCP** non va bene per LTE:

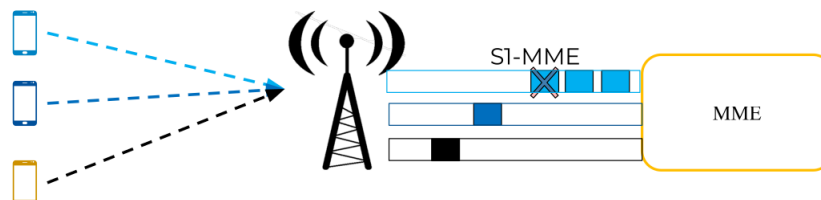
1. ha trasporto affidabile e in ordine, ma non la consegna solo affidabile e l'ordine parziale;
2. **Head of Line Blocking Problem**;
3. è **stream-oriented** e usa dei marker per delimitare i messaggi;
4. manca il **Multi-Homing**, ovvero la quadrupla (IP IP porta porta) non ci copre se un MME cade o cambia IP.

L'**HOL Blocking Problem** lo abbiamo quando dobbiamo spedire dei pacchetti. Uno dei primi viene perso e tutti quelli dopo sono bloccati nel buffer perché TCP deve aspettare per ritrasmettere.

Questo è un problema in LTE perché supponiamo di avere diversi messaggi di controllo che arrivano all'eNodeB e devono essere inoltrati all'MME. Se perdiamo un blocco di uno dei messaggi iniziali tutti gli altri non saranno processati, anche se appartengono a **dispositivi diversi** e sono totalmente ok.



Una possibile soluzione è creare **più connessioni TCP** tra eNodeB e MME, togliendo il delay che avevamo poco fa, ma questo non scala per niente visto che ci servono tantissime risorse.



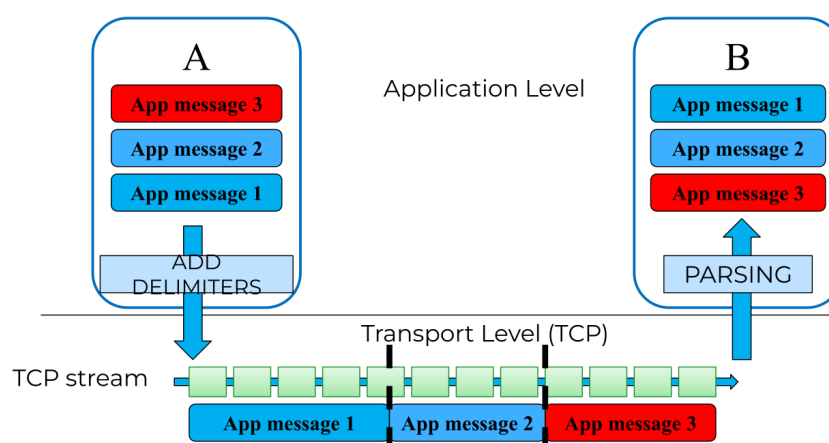
La soluzione che usa **SCTP** è quella che usano anche HTTP/2 e Quick, ovvero si aumenta leggermente l'**overhead** dell'header aggiungendo uno **stream ID**. Con questa noi abbiamo un **unico flusso SCTP** ma i pacchetti sono **taggati**, quindi anche se viene perso qualcosa viene bloccato solo quel flusso e non tutti gli altri.

Questo ovviamente introduce un **ordine parziale** nei pacchetti.

SCTP ci permette anche di avere un **pool di IP sorgente** e un **pool di IP destinazione**, visto che abbiamo un pool di MME che gestisce la nostra tracking area. Inoltre, questo ci permette la fault tolerance e il load balancing: visto che possiamo mandare i pacchetti su più path, sfruttiamo questo per parallelizzare l'invio dei pacchetti, tanto sono numerati.

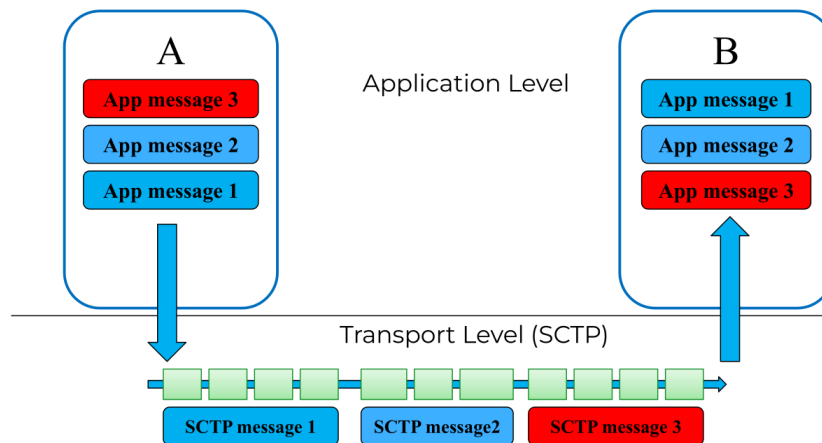
Finiamo con l'ultima differenza tra SCTP e TCP.

In **TCP** abbiamo uno **stream**, ed è compito dell'applicazione delimitare i vari messaggi. Con «delimitare» intendiamo che in un blocco dati possiamo avere i dati di più messaggi, per questo il marker è fondamentale.



Noi vogliamo eliminare il marker per evitare il parsing.

Sfruttiamo quello che fa **UDP**: ogni blocco dati è riferito ad un **messaggio solo**, quindi non serve nessun marker e nessuna divisione, è SCTP che assembla e riassembla.



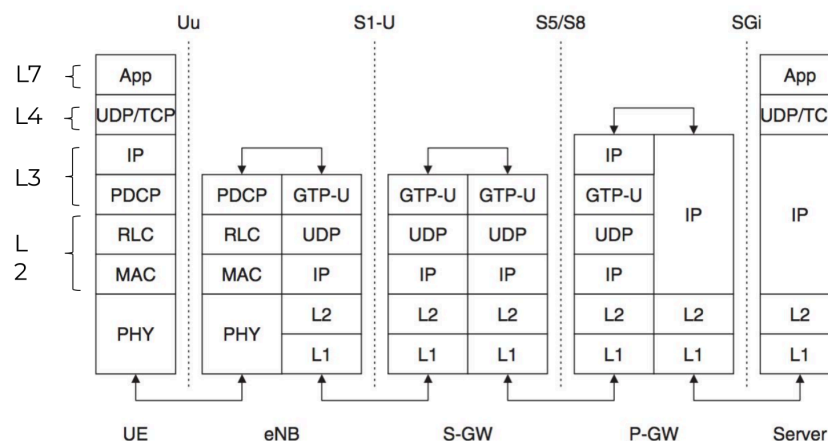
Vediamo, per riassumere, le  **differenze**  tra TCP e SCTP.

|                                        | SCTP | TCP | UDP |
|----------------------------------------|------|-----|-----|
| Message-Oriented                       | X    |     | X   |
| Byte-Oriented                          |      | X   |     |
| Connection-Oriented                    | X    | X   |     |
| Full-Duplex                            | X    | X   | X   |
| Trasporto affidabile                   | X    | X   |     |
| Consegna in ordine                     | X    | X   |     |
| Consegna NON in ordine                 | X    |     | X   |
| Controllo di flusso e di congestione   | X    | X   |     |
| ECN (Explicit Congestion Notification) | X    |     |     |
| Multi-Streaming                        | X    |     |     |
| Multi-Homing                           | X    |     |     |
| Prevenzione SYN flooding attack        | X    |     |     |

Come vediamo, siamo sia  **connection-oriented**  che  **message-oriented** , con una serie di qualità ottime a fare da contorno al protocollo.

#### 2.1.1.6. User Plane

Passiamo ora allo  **User Plane** , in cui ci sono ancora UE ed eNodeB.



In questo caso riconosciamo molti più protocolli di prima. Rimangono **PDCP** e **RLC** per il controllo errori, ma scompare **RRC** per lasciare spazio al solo **IP** con sopra la classica parte ISO/OSI. Inoltre, viene usato **UDP** e **GTP-U**, che riprendiamo dopo.

Come vediamo, tra UE ed eNodeB siamo identici, mentre tra **eNodeB** e i vari **gateway** abbiamo qualcosa di diverso.

Dal **server** in poi abbiamo il mondo esterno, che dialoga direttamente con la parte client dell'UE in maniera **end-to-end**.

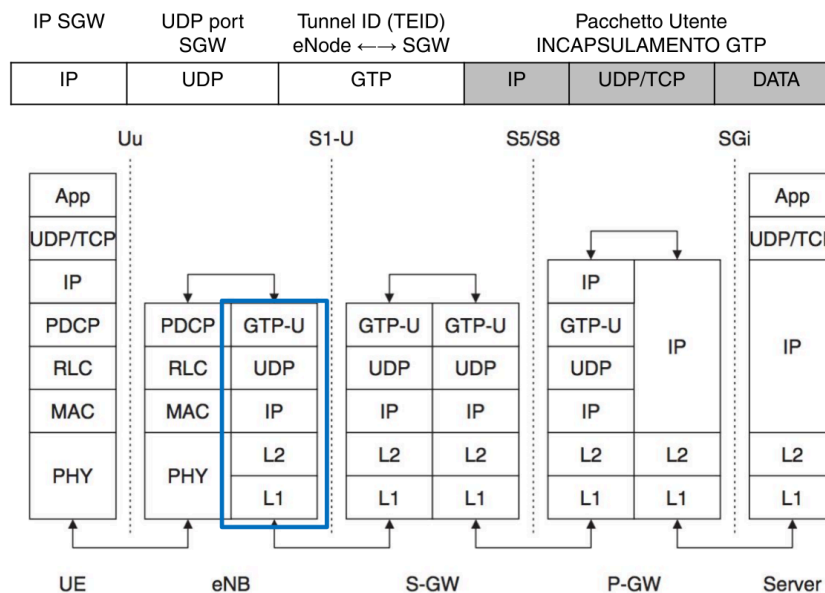
Nell'architettura abbiamo **tre livelli IP** diversi:

1. **tra UE e PGW** abbiamo indirizzi **IP interni**, assegnati da NAT e DHCP, quindi non riguardano il servizio dell'operatore;
2. **tra PGW e server** abbiamo **indirizzi IP pubblici dell'operatore mobile**, ovvero del servizio che viene offerto;
3. **tra eNodeB e gateway** abbiamo **indirizzi IP interni** che usiamo per fare routing.

Dentro la rete usiamo **GTP-User Plane** (GTP-U), che abbiamo visto l'altra volta, ma riprendiamo velocemente perché ho voglia di rifarlo.

Abbiamo un UE che ha una **Packet Data Network Session** (PDN Session), ovvero un collegamento con il PGW che deve essere mantenuto anche se l'UE si muove. In questa sessione, quando ci spostiamo, dovremmo ogni volta modificare le tabelle di routing che abbiamo tra l'UE e il PGW, facendo **overhead di controllo** gigante se cambiamo continuamente l'eNodeB a cui siamo agganciati.

**GTP** inizia proprio all'uscita di un **eNodeB** verso la rete core.



Come vediamo, andiamo ad impacchettare il pacchetto utente in un **pacchetto GTP**, che incapsula appunto il messaggio con l'IP del SGW, la porta UDP usata e il **Tunnel Endpoint ID** (TEID).

Grazie a questo escamotage le tabelle interne dei vari **router** sono **fisse**, non cambiano mai e quindi tutto quello che fa l'UE non va a modificare la struttura interna della rete.

Quando arriviamo al PGW, dopo molteplici incapsulamenti e decapsulamenti, abbiamo solo il **pacchetto utente inalterato**.



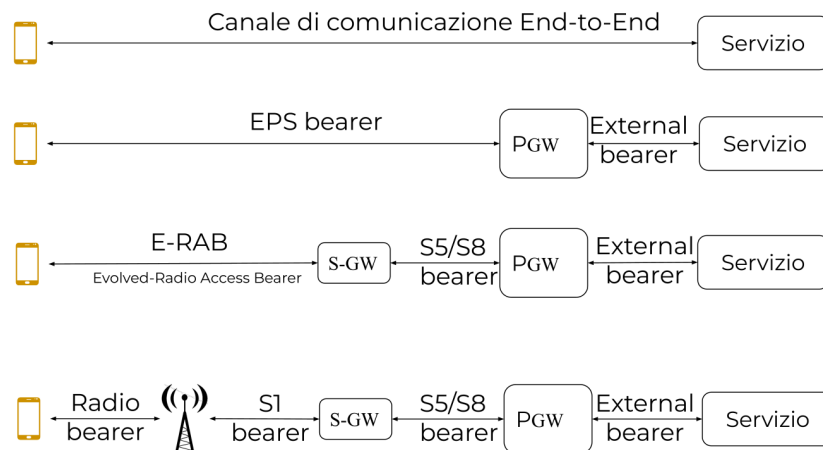
Questo vale in uplink e in downlink.

Questo **overhead** di header è bassissimo considerando la **mobilità utente**: cambiamo spesso gli eNodeB, ma le tabelle di routing dentro la rete rimangono fisse.

#### 2.1.1.7. Bearer

In LTE possiamo definire la **QoS** tramite i **Bearer**, dei canali radio che trasportano dati con una QoS ben definita.

La QoS viene definita **tra l'UE e il PGW** tramite **Evolved Packet Switch Bearer** (EPS Bearer), che viene però diviso in vari pezzi seguendo la divisione dell'architettura.



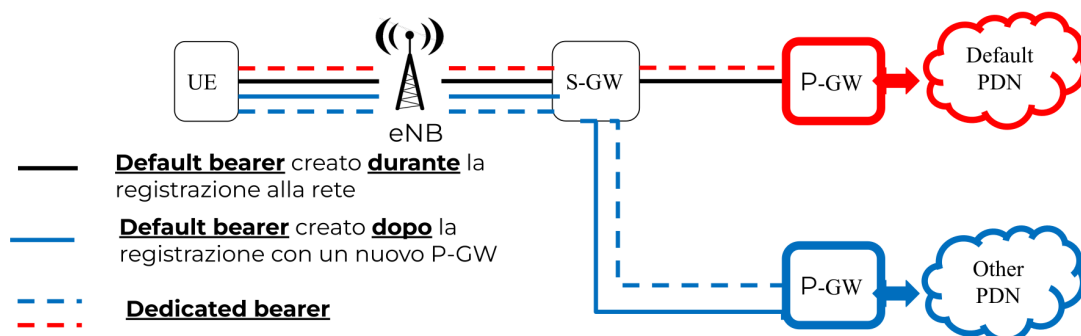
Dobbiamo essere in grado di **bilanciare** le varie parti per ottenere al massimo il delay richiesto dall'utente.

Ogni **UE** può avere al massimo 8 **bearer attivi**. Ogni UE ne ha sempre almeno uno, il **default bearer**, che viene creato dal PGW quando ci colleghiamo alla rete. Ogni bearer ha un **IP differente**.

Gli UE possono avere più **default bearer attivi**.

I **dedicated bearer** sono dei «fork» sul default bearer, in cui prendiamo l'IP del bearer da cui siamo generati. Siamo inoltre connessi allo stesso PGW, ma possiamo definire una **diversa QoS**.

Possiamo connetterci ad **altri PGW**, creando quindi altri default bearer, da cui possiamo derivare altri dedicated bearer.



Tra default e dedicated possiamo avere al massimo 8 canali attivi.

Vediamo la tabella delle varie **QoS**.

| QCI | RESOURCE TYPE | PRIORITY | PACKET DELAY BUDGET (MS) | PACKET ERROR LOSS RATE | EXAMPLE SERVICES                                                                                |
|-----|---------------|----------|--------------------------|------------------------|-------------------------------------------------------------------------------------------------|
| 1   | GBR           | 2        | 100                      | $10^{-2}$              | Conversational voice                                                                            |
| 2   | GBR           | 4        | 150                      | $10^{-3}$              | Conversational video (live streaming)                                                           |
| 3   | GBR           | 5        | 300                      | $10^{-6}$              | Non-conversational video (buffered streaming)                                                   |
| 4   | GBR           | 3        | 50                       | $10^{-3}$              | Real-time gaming                                                                                |
| 5   | Non-GBR       | 1        | 100                      | $10^{-6}$              | IMS signaling                                                                                   |
| 6   | Non-GBR       | 7        | 100                      | $10^{-3}$              | Voice, video (live streaming), interactive gaming                                               |
| 7   | Non-GBR       | 6        | 300                      | $10^{-6}$              | Video (buffered streaming)                                                                      |
| 8   | Non-GBR       | 8        | 300                      | $10^{-6}$              | TCP-based (for example, WWW, e-mail), chat, FTP, p2p file sharing, progressive video and others |
| 9   | Non-GBR       | 9        | 300                      | $10^{-6}$              |                                                                                                 |

Questa tabella contiene i **Quality of Service Class Identifiers** (QCI), che sono ID che si riferiscono ad alcune classi di servizio ben definite. Ci sono quattro **parametri** che definiscono una QoS:

1. **Minimum Guaranteed Bit Rate** (GBR), ovvero quello che la rete si impegna a darci, ovviamente se ne ha le possibilità;
2. **priorità**, usato dalle code con priorità per lo scheduling;
3. **Packet Delay Budget** (PDB) in millisecondi, che indica il delay massimo che vogliamo avere tra l'UE e il PGW;
4. **Packet Error Loss Rate** (PELR), che è la probabilità di avere un errore sui bit trasmessi.

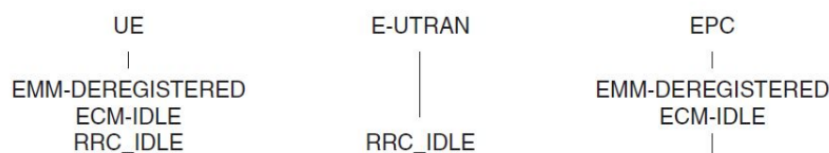
Il traffico viene **mappato sui bearer** tramite **Traffic Flow Template** (TFT), ovvero i dispositivi ricevono i dati con la QoS richiesta e mappano il flusso su uno dei canali che riesce a garantire quanto richiesto. Questo avviene se ovviamente siamo autorizzati, ovvero se abbiamo pagato.

#### 2.1.1.8. Collegamento alla rete dell'operatore

Ora vediamo la procedura di **collegamento alla rete operatore**.

Quando l'UE viene acceso siamo in:

1. **EMM\_DEREGISTERED**, dove EMM sta per **EPS Mobility Management**, e indica che non siamo registrati ad un MME, quindi la nostra mobilità non viene gestita;
2. **ECM\_IDLE**, dove ECM sta per **EPS Connection Management**, e indica che la connessione tra UE e MME per il traffico di controllo non è attiva;
3. **RRC\_IDLE**, che indica che non siamo connessi ad alcun eNodeB.

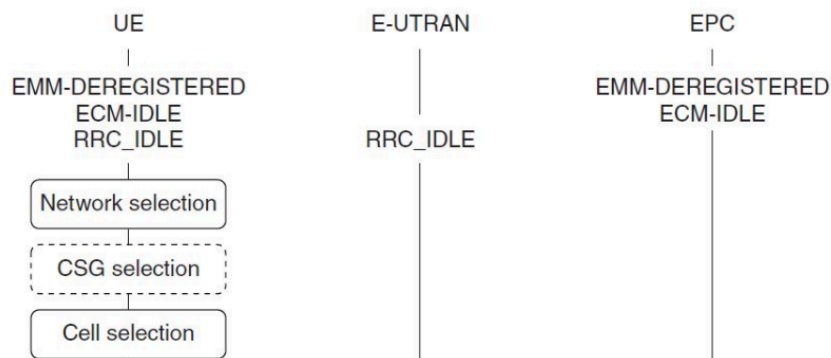


Infatti, la rete RAN **non ci conosce**.

Come prima cosa facciamo **network selection**, in cui ascoltiamo gli eNodeB per capire il loro segnale.

La **Closed Subscriber Group** (CSG) è una lista di client che sono autorizzati all'accesso della **femtocella**, ma questa procedura è opzionale come anche la sua spiegazione in questi appunti.

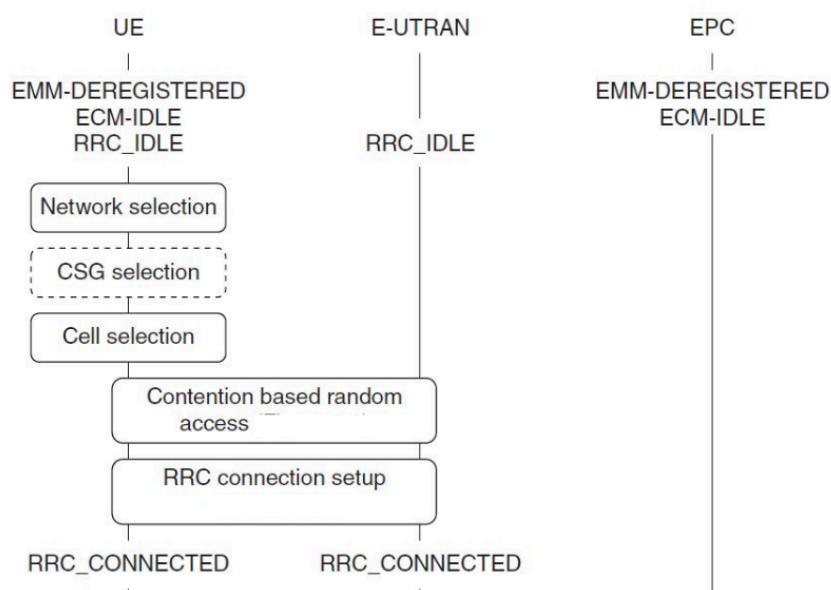
Tra tutti gli **eNodeB candidati** scegliamo quello con il **segnale migliore** facendo una **cell selection**. Alcune **metriche** per capire la cella migliore sono la potenza di trasmissione dell'UE (il mio raggio di copertura) e la potenza del segnale che riceviamo dall'eNodeB.



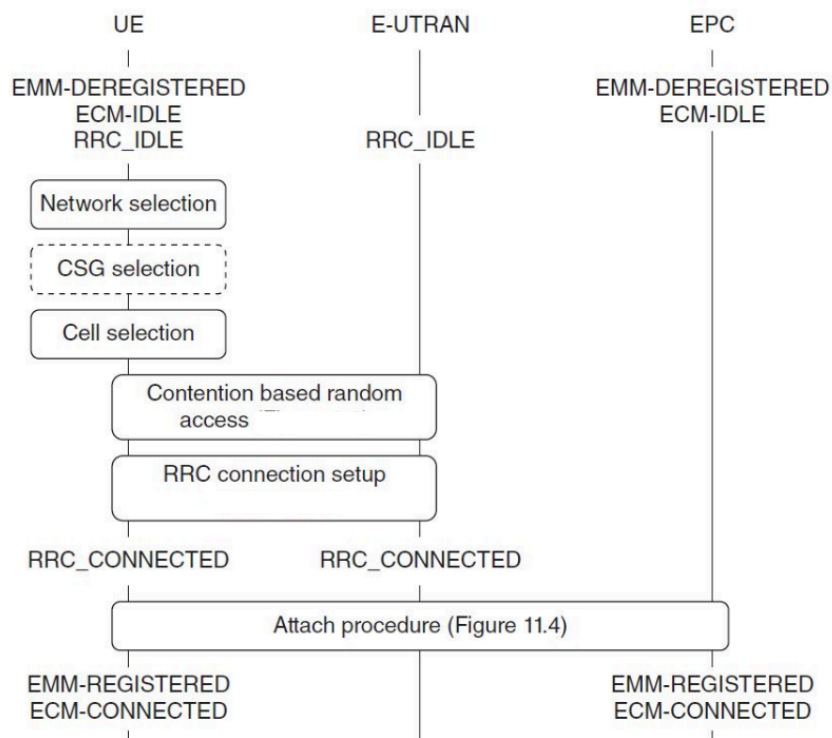
Come vediamo, non abbiamo ancora contattato nessuno.

Lo UE inizia quindi la procedura di **contesa** per l'accesso al **Random Access Channel**, ovvero lo scheduling della cella selezionata.

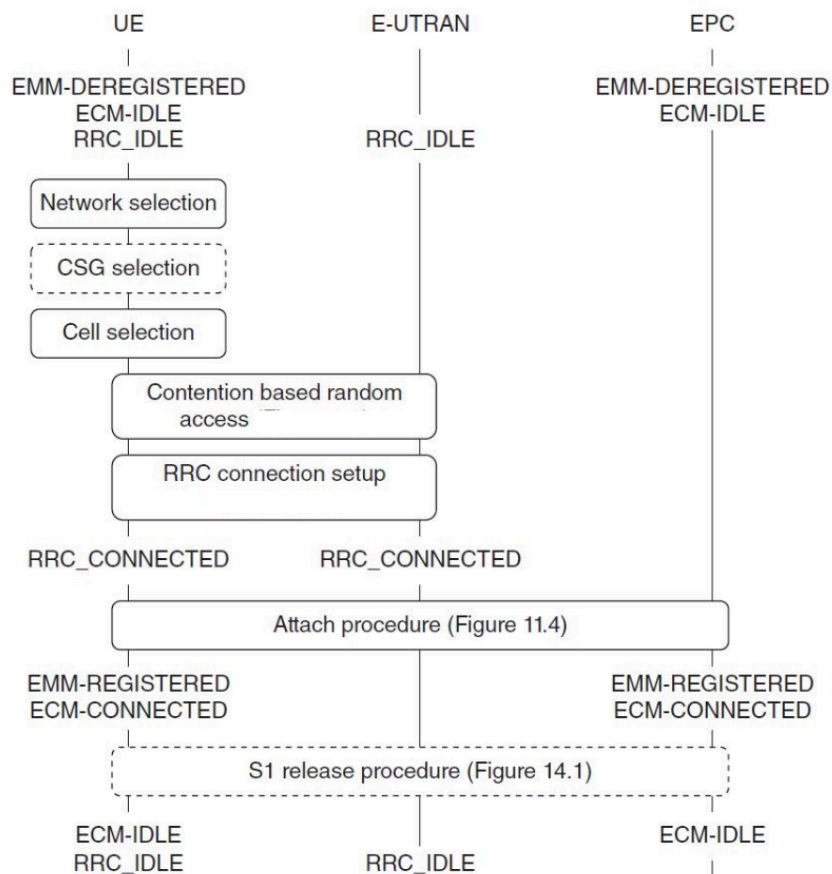
Dopo la **RRC Connection Setup**, in cui configuriamo i due livelli più bassi dello stack, siamo ufficialmente connessi all'eNodeB, ma non abbiamo ancora parlato con la **rete core**.



Con la **procedura di attachment** ci attacchiamo ad un MME, che ci rende trovabili tramite paging e abilitati per l'handover.



Se rimaniamo inattivi per un po' di tempo possiamo andare in **idle**, ma mantenendo ancora il **Mobility Management**, ovvero si riesce sempre ad avere traccia della nostra posizione, lasciamo solo i **RB** ad altri.



## 3. Lezione 13 [02/03]

### 3.1. Rete cellulare

#### 3.1.1. LTE

##### 3.1.1.1. Handover

In **LTE** avviene solo l'**hard handover**, ovvero non si ha un nodo centralizzato tra gli eNodeB, ma abbiamo due modalità possibili:

1. **seamless, senza continuità**, che presenta minore latenza ma ammette delle ritrasmissioni, ed è usato ad esempio nel **traffico VoIP e realtime**;
2. **lossless, con continuità**, che presenta maggiore latenza ma perché si riduce la perdita dei pacchetti, ed è usato per il **traffico HTTP o FTP**.

L'handover lossless si attiva durante un **flusso in download**, con un uso del **buffer** cruciale per evitare che si perda quello che veniva spedito.

In questo caso, si conosce l'eNodeB nuovo che andrà a ricevere il dispositivo, quindi iniziamo a mandare i pacchetti non ancora inviati – bufferizzati – a questo eNodeB.

Vediamo però nello specifico come funziona l'**handover**.

##### 3.1.1.1.1. Interfaccia S1

Partiamo con l'handover su **interfaccia S1**.

Si decide che deve avvenire l'**handover** tramite S1, quindi mandiamo una handover request al mio MME. La decisione è della rete, l'UE fa ben poco, è tutto a carico dell'eNodeB.

**Inoltriamo** poi la richiesta al nuovo MME, che avrà in carico il traffico di controllo, sempre tramite la S1.

Ora dobbiamo dire all'eNodeB nuovo che dovrà tenere in carico l'UE, quindi **inoltriamo** ancora l'**handover request** all'eNodeB. In questo momento conosciamo tutta la situazione attuale, soprattutto gli MME, che tengono al loro interno i **bearer attivi** per ricreare completamente la situazione nel nuovo eNodeB.

Dobbiamo prepararci per accogliere il nuovo UE, quindi facciamo il **setup delle risorse** quali slot, bearer, e tutto quello che serve per tenere il dispositivo. Come conferma all'MME nuovo mandiamo un **ack**.

Siamo pronti per ricevere, quindi facciamo avvenire l'handover: avvisiamo l'MME vecchio che siamo pronti, e lui manda all'eNodeB vecchio l'**handover command**.

In questo momento parliamo finalmente con l'**UE**, che viene avvisato dell'handover. Nel mentre che questo UE si sistema, il vecchio eNodeB ha finito il suo lavoro, e deve solo dire alla sua rete core MME di emettere un **eNodeB status transfer**, e inoltrare i dati al nuovo eNodeB se siamo nel caso **lossless**.

Un messaggio simile è l'**MME status transfer**, dal nuovo MME al nuovo eNodeB per dire che l'MME ora gestisce quel dispositivo.

Quando l'UE è **pronto per il cambio** deve notificare il nuovo eNodeB tramite una **conferma di handover**.

Infine, ultime notifiche sono l'**handover notify**, dall'eNodeB all'MME, notificando che è andato tutto a buon fine, e due messaggi tra gli MME per confermare (con ack) che le risorse sono state spostate correttamente.

Gli ultimi due messaggi in realtà sono la **Tracking Area Update Request**, che aggiorna la tracking area dell'eNodeB, e la **liberazione delle risorse** nel vecchio eNodeB.

L'handover ovviamente avviene se la rete è sicura che questo può avvenire: se non si hanno le risorse per fare l'handover questo non accade.

#### 3.1.1.1.2. Interfaccia X2

Vediamo invece l'handover su **interfaccia X2**. In questo caso, non dovendo contattare la rete core, si fa tutto tramite scambi di messaggi tra eNodeB.

Dopo una serie di **misure di controllo** i due eNodeB decidono che deve avvenire l'**handover**: prima avviene una **handover request** dall'eNodeB vecchio a quello nuovo, che deve fare un **resource setup** contattando anche l'MME, visto che è anche lui uno che riceve i messaggi.

Si manda poi un **ack** all'eNodeB vecchio per confermare che si è pronti, e come prima si manda l'**handover command** all'UE.

Passiamo quindi il comando all'eNodeB nuovo con uno **status transfer**, con anche l'UE che contatta l'eNodeB nuovo per avvisarlo che ha finito il suo setup ed è pronto con una **handover complete**. In questa fase avviene anche il forwarding dei dati su X2 se siamo in lossless.

Si parla ancora con la rete core (in realtà prima volta) per richiedere il **path switch**, che ha un ack di ritorno.

Infine, si ha il rilascio delle risorse nel vecchio eNodeB.

#### 3.1.2. Edge Computing e Network Softwarization

Per **5G** abbiamo una rappresentazione, fatta prima della standardizzazione, che rappresenta tutti gli intenti e casi d'uso che si volevano implementare. Queste sono incastonate in **classi di servizio**:

1. **Enhanced Mobile Broadband** (eMBB), che è l'evoluzione del 4G, e presenta servizi orientati alle persone, con elevata banda, HD streaming e AV/VR;
2. **Ultra-reliable and Low-latency Communications** (uRLLC), che fornisce servizi orientati alle industrie, con bassissima latenza e affidabilità, come ad esempio il controllo remoto e la guida autonoma;
3. **Massive Machine Type Communications** (mMTC), utilizzando quando si ha una alta densità di connessioni, come nelle smart cities o smart agricolture.

In questa piramide abbiamo, nei vertici, questi tre tipi di servizi appena elencati, e nel mezzo abbiamo una combinazione di questi, a formare dei **servizi ibridi**.

Abbiamo quattro direzioni principali che rappresentano gli obiettivi del 5G:

1. **maggiore efficienza spettrale**;
2. **maggiore spettro**;
3. **riuso spaziale**;
4. **softwarizzazione della rete**, permettendo una convergenza tra ICT e IT.

Le tecnologie principali introdotte in questa generazione sono:

1. **Software Defined Network** (SDN);
2. **Network Function Virtualization** (NFV);

3. **Centralized-RAN** (C-RAN) e **Virtual-RAN** (V-RAN);
4. **Mobile Edge Computing** e **Multi-access Edge Computing**;
5. **Network Slices**.

#### 3.1.2.1. SDN

Nelle **SDN** abbiamo un **controller** che permette di configurare il traffico di controllo e dati: infatti, prima le regole di forwarding e gli algoritmi di controllo erano tutte nei dispositivi, mentre ora vengono mantenute solo le regole di forwarding mentre gli algoritmi di controllo sono all'interno del controller. Ora il controller, che conosce la rete, può avere una **visione globale** e scegliere al meglio il controllo da fare.

Le SDN si sono evolute nel tempo: prima veniva configurato solo il **control plane**, lasciando il data plane fisso, mentre ora possiamo fare anche il **data plane**, arrivando fino al **livello data link**.

L'utilizzo delle SDN porta numerosi vantaggi:

1. **flessibilità** nella gestione della rete;
2. permette una **visione centralizzata** della rete, ottimizzando il **routing** visto che avviene in un solo posto;
3. semplificazione della gestione a livello applicativo;
4. il testing e la configurazione di nuovi protocolli è più semplice e veloce.

Ovviamente abbiamo alcune sfide da superare:

1. il **controller** è il punto debole dell'architettura;
2. serve una **reazione realtime** ai cambiamenti;
3. **ottimizzazione** del numero di regole;
4. la sicurezza del controller è cruciale, visto che il controllo di quello permette il controllo della rete.

#### 3.1.2.2. NFV

La **Network Function Virtualization** risolve un problema semplicissimo.

Quando compriamo l'attrezzatura paghiamo un occhio della testa le varie robe, ma magari ho una copia sola di tutto. Con questa configurazione la rete non decolla, oppure decoliamo, facciamo un successo gigante ma non possiamo **scalare** la configurazione.

L'idea delle NFV è quella di prendere il data plane, **separare** il software/firmware che implementa la funzionalità dei moduli dall'hardware e ne creiamo una versione VM o container. Ora abbiamo un HW standard sul quale io posso far girare quello che voglio in base alle esigenze.

Questo ci rende **flessibili e scalabili**, ma serve una descrizione della rete tramite **Service Function Chain** (SFC), che descrive dove e come sono implementati i servizi nella rete.

Per fare questo ci aiuta il **NFV Orchestrator**: data una richiesta di deployment di un servizio, l'orchestratore riconosce quali sono i template necessari e si tirano su le istanze di quello richiesto nel luogo migliore.

Nel basso abbiamo l'**hardware**, sul quale si ha uno **strato di virtualizzazione**, che si chiama **NFV Infrastructure**. Sopra abbiamo le effettive **network functions**, con i vari **moduli EMS di controllo**.

Ancora più sopra abbiamo BSS e OSS, che si usano per gestire i servizi, assieme all'**NFV Management and Orchestrator** (MANO), che contiene:

1. **Virtual Infrastructure Manager** (VIM), che gestisce l'infrastruttura virtuale come risorse, CPU, link, eccetera;
2. **VNF Management** (VNFM), che controlla lo stato delle singole VNF, sa quando commissionare e de-commissionare istanze, eccetera;
3. l'**orchestratore**, che è quello che fa tutto, controlla VIM e VNFM, prende decisioni sul servizio richiesto e sullo stato dell'infrastruttura.

Con le NFV abbiamo a disposizione:

1. **flessibilità massima, scalabilità**, agilità della rete e dei servizi, anche nella gestione;
2. **indipendenza** tra hardware e software;
3. rapida prototipizzazione e introduzione di nuovi servizi, per operatore e utenti finali;
4. uso delle risorse ottimizzato e condiviso.

Come prima, sono presenti anche numerose sfide:

1. le **prestazioni** devono essere comparabili a quelle con hardware dedicato, visto che nelle NFV non abbiamo la stessa velocità dato l'overhead di virtualizzazione;
2. serve una **gestione efficiente delle risorse**, tramite orchestratore e VNF manager;
3. **sicurezza**;
4. gestione della fase di transizione in cui hardware e software coesistono;
5. **gestione multi-tenant**, ovvero si hanno più operatori di servizio che condividono le stesse risorse hardware e software.

### 3.1.2.3. Cloud

Infine, in 5G abbiamo anche il **cloud**, che assieme a SDN e VNF formano quasi completamente tutta l'architettura di rete. Infatti, con le VNF noi mettiamo le varie parti dell'architettura dove vogliamo, e poi tramite SDN le colleghiamo.



## 4. Lezione14 [06/03]

### 4.1. 5G

#### 4.1.1. NFV su RAN

Le **NFV** sono usate nella parte RAN, sull'eNodeB.

Per ora come architettura abbiamo due parti:

1. **Baseband Unit** (BBU), con operazioni di gestione dal **data link** in su;
2. **Remote Radio Head** (RRH), dove si ha l'antenna effettiva al **livello fisico**.

A noi serve **alimentazione**, **condizionamento** al calore per la BBU e un **alto carico computazionale**. Per fare ciò possiamo usare l'interfaccia X2 per ridurre le interferenze, ma questa ha una **limitata visione** dello stato degli altri eNodeB ed è **mono-standard**, ovvero lavoriamo solo 4G, mentre noi vorremmo avere un multi-RAT.

Un altro problema è la **densificazione**: con essa abbiamo una copertura più piccola ma localizzata, ma questo ci obbliga a replicare BBU e RRH ovunque.

L'uso delle NFV ci permette di avere una solo RRH (parte hardware) e un **pool di BBU remoto** (vBBU), collegato tramite cavo, che contiene tutta una parte software con i vari standard.

Con questo **datacenter** è possibile una **BBU multi-tecnologia**: infatti possiamo configurare tante vBBU quante ce ne servono in quel momento. Abbiamo comunque un limite, che è il **cavo**, che ce lo dà in capacità e tempo di propagazione: dobbiamo quindi mettere vicine le RRH con le vBBU, di solito entro i 10 chilometri.

Abbiamo una serie di **vantaggi**:

1. **riduzione del CAPEX** (investimento di capitale), con una riduzione del numero di apparati e dei costi per introdurre nuove tecnologie RAT;
2. **riduzione dell'OPEX** (costo per fare funzionare), con un minore consumo energetico, l'ottimizzazione delle risorse BBU remote e la gestione dinamica della potenza delle celle;
3. **migliori prestazioni** con una migliore gestione delle interferenze e una densificazione delle celle più sostenibile;
4. possibilità di avere il **multi-RAT**.

#### 4.1.2. Cloud/Edge computing

Il **Cloud Computing**:

1. offre una serie di servizi PaaS, IaaS, SaaS, **QualsiasiCosaAAS**;
2. offre **scalabilità orizzontale** e dinamica;
3. riduce CAPEX e OPEX;
4. offre alta affidabilità e disponibilità;
5. permette ridondanza e repliche;
6. ha alti livelli di **virtualizzazione** e utilizzo hardware.

Nel cloud abbiamo infatti un **hardware standard** sul quale abbiamo **VM** e **container** ai quali accedono i vari utenti che usano la **rete internet**. Tra il cloud e gli utenti abbiamo gli ISP ma anche la **rete mobile edge** e la **rete mobile core**.

Come scenario questo va bene, ma ha una elevata latenza, jitter e questo non ci permette di avere un controllo e una gestione ottima per le **near-real app**.

Infatti, due dispositivi che stanno comunicando sullo stesso livello devono passare necessariamente dal servizio cloud, e questo porta ai problemi appena presentati.

Arriva quindi l'idea del **Multi-access Edge Computing** (MEC ETSI), che vuole **avvicinare la computazione** che avviene nel cloud all'utente.

In ciascun livello dell'architettura abbiamo una **rete geograficamente distribuita** che permette di avere della potenza di calcolo vicina all'utente. Ovviamente più andiamo verso il cloud e più abbiamo latenza e potenza di calcolo.

Per questo dobbiamo capire **dove** installare i vari moduli e anche capire come gestire la mobilità: questo dipende dai requisiti del modulo e dai link di comunicazione tra i moduli.

Abbiamo numerosi **vantaggi**:

1. l'architettura è decentralizzata e localizzata;
2. la latenza e il jitter end-to-end si riducono;
3. si riduce il traffico verso la rete core;
4. si ha un migliore supporto a servizi di location e context-aware.

Sono presenti anche numerose **sfide**:

1. integrazione nella rete per far coesistere MEC e lo standard 3GPP, e trasparenza di questo agli UE;
2. portabilità delle applicazioni MEC, con allocazione e de-allocazione trasparenti e architetture hardware e software standard e open;
3. sicurezza, che deve garantire isolamento tra le VM nei MEC server e deve controllare l'uso corretto delle risorse;
4. performance, che deve capire come dimensionare le VM e come allocarle in maniera ottimizzata all'interno della rete.

Vediamo come è questa parte dello **standard**.

Abbiamo **tre livelli**:

1. **MEC system level**, che ha una visione globale su tutti i siti MEC, è centralizzato ed ha quindi una visione di sistema;
2. **MEC host level**, che è specifico per ogni singolo datacenter distribuito;
3. parte di rete, che è esterna allo standard.

Nel MEC host level abbiamo la **virtualization infrastructure**, che virtualizza l'hardware e le MEC applications, che sono VM o container.

#### 4.1.3. Network Slices

Nel 5G abbiamo anche le **network slices**: queste sono una estensione del concetto di **radio bearer**. Prima in 3G avevano delle QoS, che poi in 4G sono state estese.

Ora, in 5G, abbiamo la possibilità di trasformare la rete da un **paradigma statico** (one size fits all) ad un **paradigma dinamico** nel quale le reti logiche sono create **on demand** con risorse e topologie ottimizzate per servire uno scopo specifico.

Una **network slice instance** è un insieme di network function fisiche e virtuali con risorse di rete che sono organizzate e configurate per fornire una rete logica che soddisfa certe caratteristiche.

#### 4.1.4. Architettura

Ci ricordiamo l'architettura di 4G, passiamo a quella del 5G.

Volevamo una rete core che potesse funzionare anche con la **rete edge e cloud contemporaneamente**.

Vengono quindi creati dei PGW classici e dei PGW per edge, ma che sono chiamati nello stesso modo. Abbiamo infatti un solo modulo **User Plane Function** (UPF) che è connesso agli UE, e per fare la classificazione del traffico abbiamo un **Uplink Classifier** (ULCL), che appunto decide dove inviare il traffico dall'UE verso il cloud o la rete classica.

Gli UPF possono anche comunicare tra di loro tramite interfaccia N9.

Nella parte del **control plane** abbiamo una VNF per ogni componente, che comunicano tra loro tramite un **protocollo publish-subscribe** che si basa sulle API REST.

Abbiamo questo approccio perché non possiamo cambiare tutta l'architettura 4G e dobbiamo essere in grado di **riutilizzare**, non possiamo andare stand-alone.

#### 4.1.4.1. Control plane

Vediamo qualche modulo nello specifico.

Il **Network Repository Function** (NRF) permette di registrarsi come entità nella rete, così che poi gli altri possono trovare e utilizzare il mio servizio tramite API.

L'MME viene diviso in:

1. **Access & Mobility Management Function** (AMF) che gestisce la maggior parte del traffico di segnalazione per autenticazione, registrazione e modalità. Inoltre, non gestisce la sessione dati;
2. **Session Management Function** (SMF) che gestisce appunto la sessione dati.

L'**Unified Data Management Function** (UDM) è un fronted unico per gestire i database.

Il **Policy Control Function** (PCF) è il vecchio PCRF e controlla le policy e chi può fare cosa.

L'**Authentication Server Function** (AUSF) gestisce tutto ciò che riguarda l'autenticazione e la generazione delle chiavi di cifratura.

Il **Network Slice Selection Function** (NSSF) gestisce la selezione delle slice tra UE e quelle ammesse, che sono contenute nell'UDM.

La **Network Exposure Function** (NEF) e la **Application Function** (AF) permettono alla rete e alle applicazioni/servizi esterni di dialogare tra loro esponendo le proprie funzionalità. In poche parole, permettiamo a servizi esterni di poter **entrare nella rete** dell'operatore tramite interfacce standard per fare alcune funzioni come il controllo e la gestione degli accessi ai servizi.

Nella prossima immagine vediamo anche l'evoluzione che abbiamo avuto dal 3G al 5G, vedendo un continuo **snellimento** dei vari plane.

#### 4.1.4.2. User plane

L'architettura viene definita a **livelli**, che sono più semplici, visto che abbiamo un singolo **modulo UPF** e diverse istanze che possono dialogare tra loro.

Possono essere attive più data network contemporaneamente tramite **bearer**.

La classificazione avviene solo **Uplink** perché nel Downlink noi andiamo sempre verso l'UE, mentre in Uplink dobbiamo capire l'UE dove vuole mandare il pacchetto.

Vediamo brevemente un esempio di **Network Slices**.

Come vediamo, sono molto eterogenee: ogni slice dice una tipologia di servizio che si vuole offrire, con le risorse che si possono mettere, e tramite VNF possiamo comporre il servizio di rete su quella risorsa.

## 5. Lezione 15 [10/03]

### 5.1. 5G

#### 5.1.1. Network Slice

Con le **Network Slices** estendiamo il concetto di **Bearer**, rendendoci quindi estremamente versatili e flessibili. Le slice sono identificate da un **ID** di 8 bit, che specifica il tipo di slice/service. Questi primi 8 bit sono la **tipologia**, poi un operatore può definire singole classi di slice usando altri 24 bit.

Il dialogo tra UE, AMF, NSSF e UDM permette di determinare quello che è consentito ad un UE durante una richiesta di Slice.

#### 5.1.2. Integrazione con Edge Computing

L'integrazione del **MEC** in 5G avviene tramite VNF e **Physical Network Function** (PNF).

Vediamo come funziona con un esempio.

Un UE ha un bearer che lo collega ad un UPF per una data DN. Viene ora richiesto un nuovo servizio edge: l'**orchestratore** trova l'edge host, la **MEC application** configura il servizio richiesto e attraverso il **SMF** viene creato un nuovo UPF con anche l'uplink classifier.

I vari MEC host possono essere messi in diverse **topologie**:

1. il MEC host è sul **sito radiomobile**, con una bassissima latenza per fare processing locale dei dati, ma non va benissimo perché ha copertura e risorse limitate;
2. il MEC host è nel **ring di accesso alla rete**, con gli eNodeB collegati ad anello, che permette bassa latenza e processing locale ma siamo comunque ancora limitati;
3. il MEC host è nella **rete backhaul** ma non nella rete core, quindi copriamo un'area metropolitana;
4. il MEC host è nella **rete core**, si ha latenza (meglio del cloud btw) ma abbiamo a disposizione tantissime risorse.

#### 5.1.3. Latenza

Uno degli obiettivi del 5G era avere una **latenza inferiore al millisecondo**. Purtroppo, non possiamo usare la rete 4G perché i **resource block** (RB) di 4G erano esattamente di durata 1ms.

Viene scelto quindi di cambiare la parte radio, creando la **5G New Radio** (5G NR), in cui trasmettiamo lo stesso data rate di prima (14 simboli in OFDMA) in meno tempo.

Ricordiamo che una durata del simbolo dipende dal sub-carrier spacing, quindi più un simbolo dura e più dobbiamo distanziare i simboli. Come soluzione riduciamo quindi la durata dei simboli, per avere più spazio e più banda.

Lo **standard 5G NR** definisce 5 durate, indicate come **numerology** e numerate da 0 a 4. Abbiamo anche due **intervalli di frequenze** FR1 e FR2, uno classico e uno usato quando ci serve tanta banda.

Abbiamo quindi una tabella che indica, per ogni numerology, quanto sono distanti le sotto-portanti. Ci viene poi detto in che bande FR possiamo usare quelle numerology, visto che nelle FR1 non abbiamo tanto spazio libero.

Per ogni numerology, viene indicato poi spacing, durata del simbolo, durata del prefisso ciclico e quanto dura tutto il simbolo in totale.

Infine, grazie a queste informazioni, viene indicato quanto deve essere ampio un RB per contenere le sub-carrier.

Avendo quindi diverse dimensioni da scegliere abbiamo uno **scheduling** molto più complicato ma che ci permette di incastrare tanti blocchi in maniera ottimale.

#### 5.1.4. Architettura Standalone e Non-Standalone

Nel caso **Standalone** abbiamo l'eNodeB 4G e la rete core 4G, oppure lo stesso ma full 5G, oppure una situazione ibrida che mischia le due.

Come vediamo, abbiamo tantissime configurazioni possibili.

Inoltre, ora l'eNodeB può essere decomposto in **Central Unit** e **Distributed Units** per poter scalare il numero di eNodeB. Questo è quello che avviene nello standard **OPEN-RAN**.

## 5.2. Rete satellitare

Finiamo il nostro corso con la **rete satellitare**.

Il **piano orbitale** è un piano inclinato di un certo angolo rispetto alla linea dell'**equatore**. Su questo piano girano i satelliti. Le **costellazioni** sono un insieme di piani orbitali.

La **geometria** del link satellitare si articola in tre punti:

1. **angolo azimuth**, che è rispetto al nord geografico in senso orario, ed è quello che ci orienta verso il satellite;
2. **angolo di elevazione**, che è rispetto all'orizzonte e ha il **punto di Zenit** a 90 gradi;
3. **angolo di copertura**, che indica quanto copriamo della superficie terrestre con il satellite.

Se prima eravamo noi in movimento, ora è il satellite che può variare di molto la distanza del nostro link (quando non siamo con satelliti geostazionari).

Un altro aspetto da considerare l'**attenuazione** del segnale in funzione dell'angolo di elevazione: infatti, avendo poca elevazione dobbiamo attraversare l'atmosfera storti, e quindi abbiamo più effetti di **assorbimento atmosferico**.

### 5.2.1. Orbite

Abbiamo tre tipi di orbite:

1. orbita **Geostationary Earth Orbit** (GEO), la più lontana, che ha durata di orbita di esattamente un giorno e ruota come la terra. Si trova oltre i 35k chilometri e ha qualità del segnale bassa con elevato delay;
2. orbita **Low Earth Orbit** (LEO), la più vicina, tra i 500 e 1500 chilometri di altitudine. Presenta basso delay e uso migliore dello spettro, ma la copertura richiede di fare handover visto che il periodo di orbita è molto breve;
3. orbita **MEO** (Medium Earth Orbit), che ha una posizione che prende il meglio e il peggio delle due presentate prima.

Come vediamo in questo grafico, nelle orbite basse abbiamo delay, periodi e coperture piccole, mentre nelle orbite più alte tutti questi fattori crescono.

Nelle GEO in realtà abbiamo anche le **Geosynchronous Earth Orbit** (GSO), che hanno una traiettoria che forma un 8 nel cielo.

Sulle orbite LEO e MEO ci serve una **costellazione** di satelliti per garantire una copertura continua globale – o almeno regionale.

### 5.2.2. Architettura

L'**architettura** della rete satellitare è formata da tre segmenti:

1. **space segment**, che sono i satelliti e le costellazioni con l'**Inter-satellite link**;
2. **ground segment**, che ha due sotto-blocchi:
  - una parte gestisce lo space segment tramite telemetria, allineamento e tracking;
  - una parte fa da **gateway** per uscire dalla rete satellitare;
3. **user segment**, che sono gli **utenti fissi** (antenne/parabole) e **mobili** (UE che cammina) che usano i servizi della rete.

La **comunicazione satellitare** avviene uplink dalla stazione al satellite, e poi in downlink con il percorso opposto. L'area di copertura è maggiore del wireless terrestre, a patto di avere del delay in più. Possibile fare anche la **comunicazione broadcaster**.

Il **livello MAC** funziona tramite periodi di **contention-free** con:

1. **TDMA**, che richiede grande sincronizzazione e potenza;
2. **FDMA**, che però ha grande effetto doppler;
3. **CDMA**, che ora è usato da quasi tutti, in cui ogni satellite ha un codice;
4. **OFDMA**, che anche lui ha il doppler ma con 5G-NTN si riesce ad utilizzare, come fa ad esempio Starlink.

### 5.2.3. NTN

La **Integrated Satellite-Terrestrial Networks** si chiedono come includere la rete cellulare nel mondo satellitare, ovvero si chiedono se sia possibile avere un satellite che fa anche da eNodeB.

Le motivazioni sono molteplici:

1. dare connettività in aree remote dove la rete cellulare non arriva;
2. dare continuità di servizio;
3. migliorare l'affidabilità della rete in caso di catastrofi naturali;
4. aumentare la scalabilità della rete.

L'integrazione avviene tramite un **relay**, che fa da ripetitore del segnale aumentandone la qualità al livello fisico, così che gli UE possano ricevere il segnale.

Possibile anche usare il **link di backhaul**, ovvero il satellite connette gli eNodeB alla rete core.

Infine, è possibile anche il **direct access**, in cui il satellite offre connettività direttamente ai dispositivi.

Possiamo fare poi come in 5G, ovvero dividere la CU (a terra) e le varie DUs (sui satelliti).